

PROGRAMAÇÃO DE DISPOSITIVOS MÓVEIS

Prof. Tiago Piperno Bonetti
bonetti@prof.unipar.br



Adicionando Estilo

Com o React Native, você estiliza seu aplicativo usando JavaScript.

Todos os componentes principais aceitam um prop nomeado style.

Os nomes e valores de estilo **geralmente** correspondem ao modo como o **CSS** funciona na web, exceto que os nomes são escritos usando **CamelCase**, por exemplo, backgroundColor em vez de background-color.

O prop style pode ser um objeto JavaScript simples. Você também pode passar um *array* de estilos - o último estilo do array tem precedência, então você pode usar isso para herdar estilos.

À medida que a complexidade de um componente aumenta, geralmente é mais limpo usá-lo StyleSheet.create para definir vários estilos em um só lugar.

```

1  import React from 'react';
2  import { StyleSheet, Text, View } from 'react-native';
3
4  const App = () => {
5    return (
6      <View style={styles.container}>
7        <Text style={styles.red}>Apenas red</Text>
8        <Text style={styles.bigBlue}>Apenas bigBlue</Text>
9        <Text style={[styles.bigBlue, styles.red]}>bigBlue, depois red</Text>
10       <Text style={[styles.red, styles.bigBlue]}>red, depois bigBlue</Text>
11     </View>
12   );
13 };
14
15 const styles = StyleSheet.create({
16   container: {
17     marginTop: 50,
18     marginLeft: 20,
19   },
20
21   bigBlue: {
22     color: 'blue',
23     fontWeight: 'bold',
24     fontSize: 30,
25   },
26
27   red: {
28     color: 'red',
29   },
30 });

```

Apenas red

Apenas bigBlue

bigBlue, depois red

red, depois bigBlue

Mais em: <https://reactnative.dev/docs/text>

Estado

```
1  import React, { useState } from 'react';
2  import { StyleSheet, Text, View, Button } from 'react-native';
3
4  const Contador = (props) => {
5    const [contador, setContador] = useState(0);
6
7    return (
8      <View>
9        <Text style={styles.bigBlue}>
10         | Você clicou { contador } vez(es) no botão
11        </Text>
12
13        <Button
14         |   onPress={ () => { setContador(contador + 1) } }
15         |   title={"Clique"}
16        </>
17      </View>
18    );
19  };
```

Você clicou 3 vez(es) no botão

CLIQUE

Estado

Quantidade: 0

+

-

```
1  import React, { useState } from 'react';
2  import { StyleSheet, Text, View, Button } from 'react-native';
3
4  const QuantidadeProduto = (props) => {
5    const [quantidade, setQuantidade] = useState(0);
6
7    return (
8      <View>
9        <Text style={styles.bigBlue}>
10          Quantidade: { quantidade }
11        </Text>
12
13        <Button
14          onPress={ () => { setQuantidade(quantidade + 1) } }
15          title={"+"}
16        />
17
18        <Button
19          onPress={ () => { setQuantidade(quantidade - 1) } }
20          disabled={ quantidade == 0 }
21          title={"-"}
22        />
23      </View>
24    );
25  };
```

Visualizações de lista

O React Native fornece um conjunto de componentes para apresentar listas de dados (*FlatList* ou *SectionList*)

O componente *FlatList* exibe uma **lista** de rolagem de **dados variáveis**. Funciona bem para longas listas de dados, em que o número de itens pode mudar com o tempo.

O componente *FlatList* requer duas propriedades: *data* e *renderItem*.

data é a fonte de informação da lista.

renderItem pega um item da fonte e retorna um componente formatado para renderizar.

```

1 //exportando um array com os alunos
2 export default [
3   {ra: 1, nome: 'Felipe', nota: 80},
4   {ra: 2, nome: 'João', nota: 60},
5   {ra: 3, nome: 'Roberto', nota: 100},
6   {ra: 4, nome: 'Mariana', nota: 90},
7   {ra: 5, nome: 'Joaquim', nota: 75},
8 ]

```

```

1 import React, { useState } from 'react';
2 import { FlatList, StyleSheet, Text, View } from 'react-native';
3
4 import ListaAlunos from './alunos/ListaAlunos';
5 // importando os dados dos alunos
6
7 const App = () => {
8   return (
9     <View style={styles.container}>
10       <FlatList
11         data={ListaAlunos}
12
13         renderItem={({ item }) => {
14           return (
15             <Text style={styles.item}>
16               Nome: {item.nome} / Nota: {item.nota}
17             </Text>
18           )
19         }}
20       </FlatList>
21     </View>
22   );
23 };
24

```

Manipulação de entrada de texto

```
1  import React, { useState } from 'react';
2  import { TextInput, StyleSheet, Text, View } from 'react-native';
3
4  const App = () => {
5    const [texto, setTexto] = useState('');
6
7    return (
8      <View style={styles.container}>
9        <TextInput
10          style={{height: 40}}
11          placeholder="Digite aqui sua frase"
12          onChangeText={texto => setTexto(texto)}
13          defaultValue={texto}
14        />
15        <Text style={{fontSize: 30}}>
16          Você digitou: {texto}
17        </Text>
18      </View>
19    );
20  }
```

TextInput é um componente core que permite ao usuário inserir texto.

Possui uma propriedade **onChangeText** que leva uma função a ser chamada toda vez que o texto é alterado.

Layout com Flexbox

Um componente pode especificar o layout de seus filhos usando o algoritmo Flexbox.

O Flexbox foi projetado para fornecer um **layout consistente** em **diferentes tamanhos** de **tela**.

Você normalmente usará uma combinação de *flexDirection*, *alignItems* e *justifyContent* para alcançar o layout certo.

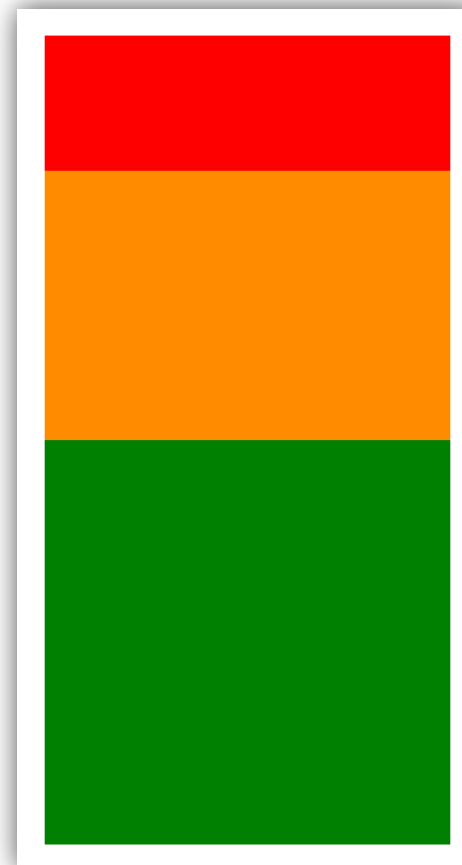
flex irá definir como seus itens irão “preencher” o espaço disponível ao longo de seu eixo principal.

O espaço será dividido de acordo com a propriedade flex de cada elemento.

```

1  import React from "react";
2  import { StyleSheet, Text, View } from "react-native";
3
4  const Flex = () => {
5    return (
6      <View style={{
7        flex: 1,
8        flexDirection: "column",
9        padding: 10,
10     }}>
11
12        <View style={{ flex: 1, backgroundColor: "red" }} />
13        <View style={{ flex: 2, backgroundColor: "darkorange" }} />
14        <View style={{ flex: 3, backgroundColor: "green" }} />
15
16      </View>
17    );
18  };

```



As visualizações vermelha, amarela e verde são todas **filhas** na visualização do contêiner que foi configurada `flex: 1`.

A visão vermelha usa `flex: 1`, a visão amarela usa `flex: 2` e a visão verde usa `flex: 3`.

$1 + 2 + 3 = 6$, o que significa que a visualização **vermelha** obterá **1/6** do espaço, a **amarela** **2/6** do espaço e a **verde** **3/6** do espaço.

Direção do Flex

flexDirection controla a direção na qual os filhos de um nó são dispostos. Isso também é conhecido como **eixo principal**. O eixo cruzado é o eixo perpendicular ao eixo principal, ou o eixo no qual as linhas de quebra são dispostas.

column (valor padrão): Alinha os filhos de cima para baixo.

row: Alinha os filhos da esquerda para a direita.

column-reverse: Alinha os filhos de baixo para cima.

row-reverse: Alinha os filhos da direita para a esquerda.

Direção do layout

A direção do layout especifica a direção em que os filhos e o texto em uma hierarquia devem ser dispostos. A direção do layout também afeta a qual borda de referência *start* e *end*.

Por **padrão**, o React Native apresenta layouts com direção de **layout LTR**. Neste modo, ***start*** refere-se à **esquerda** e ***end*** à **direita**.

LTR (valor padrão): O texto e os filhos são dispostos **da esquerda para a direita**. A margem e o preenchimento aplicados ao início de um elemento são aplicados no lado esquerdo.

RTL: O texto e os filhos são dispostos **da direita para a esquerda**. A margem e o preenchimento aplicados ao início de um elemento são aplicados no lado direito.

```

1  import React from "react";
2  import { StyleSheet, Text, View } from "react-native";
3
4  const Flex = () => {
5    return (
6      <View style={{
7        flex: 1,
8        flexDirection: "row",
9        padding: 10,
10       direction: 'rtl',
11       //direction: 'ltr',
12     }}>
13
14       <View style={{ backgroundColor: "red", height: 50, width: 50 }} />
15       <View style={{ backgroundColor: "darkorange", height: 50, width: 50 }} />
16       <View style={{ backgroundColor: "green", height: 50, width: 50 }} />
17
18     </View>
19   );
20 };

```



Justify Content

justifyContent descreve como alinhar os filhos dentro do eixo principal de seu contêiner.

flex-start (valor padrão): Alinha os filhos de um contêiner ao início do eixo principal do contêiner.

flex-end: Alinha os filhos de um contêiner ao final do eixo principal do contêiner.

center: Alinha os filhos de um contêiner no centro do eixo principal do contêiner.

space-between: Distribua uniformemente os filhos no eixo principal do contêiner, distribuindo o espaço restante entre os filhos.

space-around: Distribua uniformemente os filhos no eixo principal do contêiner, distribuindo o espaço restante em torno dos filhos. Em comparação com *space-between*, usar *space-around* resultará na distribuição do espaço no início do primeiro filho e no final do último.

space-evenly: Distribua uniformemente os filhos dentro do contêiner de alinhamento ao longo do eixo principal. O espaçamento entre cada par de itens adjacentes, a borda inicial principal e o primeiro item, e a borda final principal e o último item, são exatamente iguais.

<https://snack.expo.io/@tpipernob/justify-content>

```
1  import React from "react";
2  import { StyleSheet, Text, View } from "react-native";
3
4  const Flex = () => {
5    return (
6      <View style={{
7        flex: 1,
8        flexDirection: "column",
9        padding: 10,
10       justifyContent: "center",
11     }}>
12
13       <View style={{ backgroundColor: "red", height: 50, width: 50 }} />
14       <View style={{ backgroundColor: "darkorange", height: 50, width: 50 }} />
15       <View style={{ backgroundColor: "green", height: 50, width: 50 }} />
16       <View style={{ backgroundColor: "blue", height: 50, width: 50 }} />
17
18     </View>
19   );
20 }
```



Alinhar itens

alignItems descreve como alinhar os filhos ao longo do eixo transversal de seu contêiner. Muito semelhante a *justifyContent*, mas em vez de se aplicar ao eixo principal, *alignItems* aplica-se ao **eixo cruzado**.

stretch (valor padrão): Estique os filhos de um contêiner para coincidir com o height eixo cruzado do contêiner. Para ter efeito, os filhos não devem ter uma dimensão fixa ao longo do eixo secundário.

flex-start: Alinha os filhos de um contêiner com o início do eixo cruzado do contêiner.

flex-end: Alinha os filhos de um contêiner ao final do eixo cruzado do contêiner.

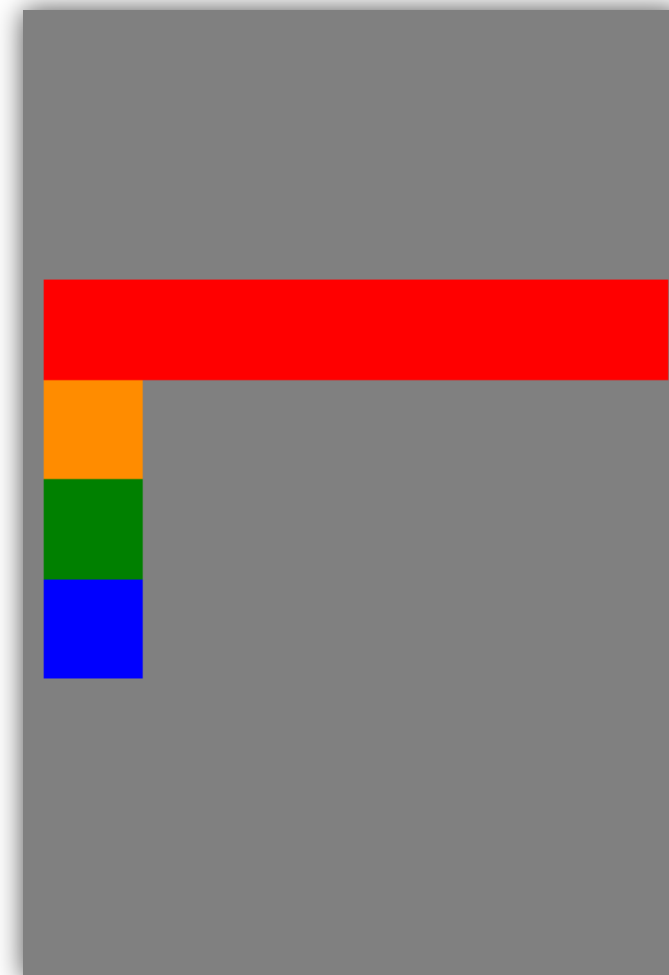
center: Alinha os filhos de um contêiner no centro do eixo cruzado do contêiner.

baseline: Alinha os filhos de um contêiner ao longo de uma linha de base comum. Cada filho pode ser definida como a linha de base de referência para seus pais.


```

1  import React from "react";
2  import { StyleSheet, Text, View } from "react-native";
3
4  const Flex = () => {
5    return (
6      <View style={{
7        flex: 1,
8        flexDirection: "column",
9        padding: 10,
10       justifyContent: "center",
11       width: '100%',
12       backgroundColor: "gray",
13       alignItems: 'stretch',
14     }}>
15
16       <View style={{ backgroundColor: "red", height: 50, width: 'auto' }} />
17       <View style={{ backgroundColor: "darkorange", height: 50, width: 50 }} />
18       <View style={{ backgroundColor: "green", height: 50, width: 50 }} />
19       <View style={{ backgroundColor: "blue", height: 50, width: 50 }} />
20
21     </View>
22   );
23 };

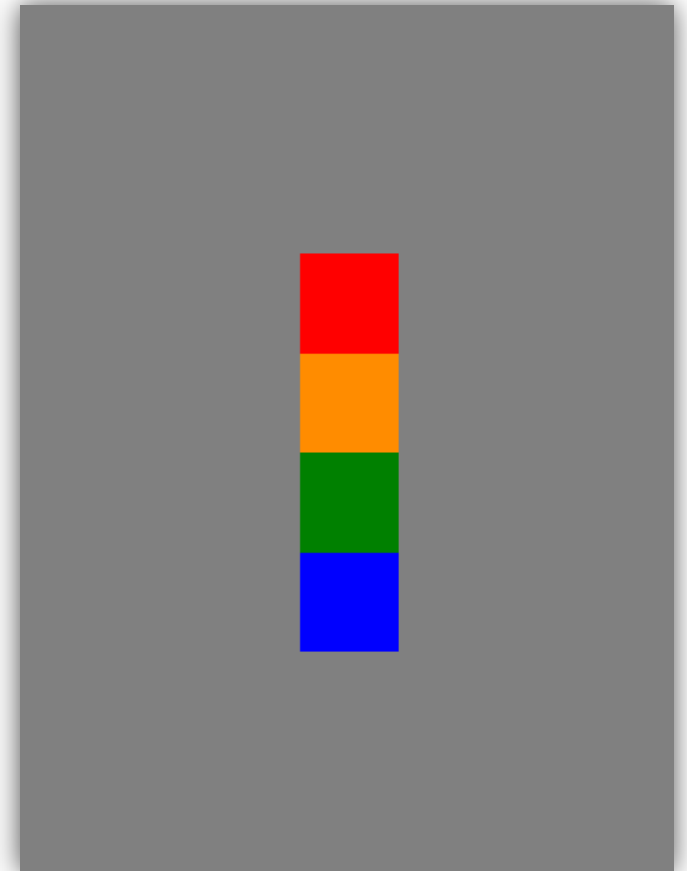
```



```

1  import React from "react";
2  import { StyleSheet, Text, View } from "react-native";
3
4  const Flex = () => {
5    return (
6      <View style={{
7        flex: 1,
8        flexDirection: "column",
9        padding: 10,
10       justifyContent: "center",
11       width: '100%',
12       backgroundColor: "gray",
13       alignItems: 'center',
14     }}>
15
16       <View style={{ backgroundColor: "red", height: 50, width: 50 }} />
17       <View style={{ backgroundColor: "darkorange", height: 50, width: 50 }} />
18       <View style={{ backgroundColor: "green", height: 50, width: 50 }} />
19       <View style={{ backgroundColor: "blue", height: 50, width: 50 }} />
20
21     </View>
22   );
23 };

```



Referências

Escudelario, Bruna de Freitas; Pinho, Diego Martins. **React Native: Desenvolvimento de aplicativos mobile com React.** Casa do código.

Documentação React Native. Disponível em: <https://reactnative.dev/docs>